Ders 6 Genişletilmiş Notu: Bağlamdan Bağımsız Gramerler, Türetme, Türetme Ağaçları ve Belirsizlik

Ders 6 Genişletilmiş Notu: Bağlamdan Bağımsız Gramerler, Türetme, Türetme Ağaçları ve Belirsizlik

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Düzenli Dillerden Bağlamdan Bağımsız Dillere
- 2. Gramer Fikri: İngilizce Benzetmesi
- 3. İlk Örnek: $S \to aSb \mid \varepsilon$
- 4. İki Kısa Gösterim
- 5. Bağlamdan Bağımsız Gramerin Biçimsel Tanımı
- 6. Gramerin Dili ve Bağlamdan Bağımsız Dil
- 7. Örnek: Çift Uzunluklu Palindromlar
- 8. Örnek: Eşleşen Parantezler
- 9. Türetme Sırası: En Soldan ve En Sağdan Türetme
- 10. Türetme Ağaçları
- 11. Belirsizlik
- 12. Belirsizlik Örneği: Aritmetik İfadeler
- 13. Tanıma Dönüş: Ağaç mı, Türetme mi?
- 14. Belirsizlik Örneği: Sarkan Else
- 15. Belirsizliği Gidermek
- 16. Giderilemeyen Belirsizlik: Doğası Gereği Belirsiz Diller
- 17. Bir Dilin Bağlamdan Bağımsız Olduğunu Göstermek

1. Düzenli Dillerden Bağlamdan Bağımsız Dillere

Bu hafta bağlamdan bağımsız dillere (context-free languages, kısaca CFL) başlıyoruz. Son bir iki derste düzenli dilleri (regular languages) öğrendik. O sırada düzenli olmayan diller de olduğunu söylemiştik. İki örnek vermiştik:

$$\{a^n b^n \mid n \geq 0\} \qquad \text{ve} \qquad \{ww^R\}$$

İkinci dil, bir w dizgisi ile onun tersinin (reverse) art arda eklenmesinden oluşur. Bu diller düzenli değildir, ama bağlamdan bağımsızdır. Yani bağlamdan bağımsız diller kümesinin içindedirler.

Bağlamdan bağımsız diller iki yolla tanımlanır: bağlamdan bağımsız gramerlerle (context-free grammars, CFG) ya da aşağı itmeli otomatlarla (pushdown automata, PDA). İkisini de göreceğiz. Önce gramerlerle başlıyoruz, aşağı itmeli otomatlar sonra gelecek.

2. Gramer Fikri: İngilizce Benzetmesi

Bağlamdan bağımsız gramerler, insan dillerinin gramerlerine bir ölçüde benzer. Gramerler dilleri ifade eder. Hoca bunu İngilizceden küçük bir parçayla gösterdi.

2.1 Küçük Bir Gramer

Bu gramere göre bir cümle (sentence), bir isim öbeği (noun phrase) ve bir yüklemden (predicate) oluşur. İsim öbeği bir tanımlık (article) ve bir isimden (noun) oluşur. Yüklem bir fiilden (verb) oluşur. Bu değişkenlerin neyden oluştuğunu, uç simgelere varana kadar yazmayı sürdürürüz:

- `<sentence>` → `<noun-phrase>` `<predicate>`
- `<noun-phrase>` → `<article>` `<noun>`
- `<predicate>` → `<verb>`
- `<article>` → a | the
- `<noun>` → cat | dog
- `<verb>` → runs | sleeps

Bu çok küçük bir gramerdir. Hocaya göre gerçek İngilizce grameri muhtemelen çok çok büyüktür, ama bu kadarıyla bile dizgi türetebiliriz (derive).

2.2 Bir Cümle Türetmek

Her türetmede başlangıç değişkeninden (initial variable) başlanır. Bu gramerde başlangıç değişkeni `<sentence>`'tır.

> **Hocanın vurgusu:** Bütün türetmeler ilk değişkenden, yani başlangıç değişkeninden başlamak zorundadır.

`<sentence>` için tek kural vardır, bu yüzden ilk adımda seçim yoktur. Sonra `<noun-phrase>` yerine `<article>` `<noun>`, `<predicate>` yerine `<verb>` yazılır:

```
<sentence> => <noun-phrase> <predicate>
           => <article> <noun> <predicate>
           => <article> <noun> <verb>
```

Artık seçim noktasına geldik. `<article>` yerine "a" ya da "the" yazabiliriz. Seçimimize göre farklı bir dizgi elde ederiz. "the" seçelim. `<noun>` için "dog", `<verb>` için "runs" seçersek "the dog runs" dizgisi çıkar. Bu dizgi, bu dilin bir dizgisidir.

Farklı seçimlerle farklı dizgiler elde edilir. "the" yerine "a", "dog" yerine "cat", "runs" yerine "sleeps" seçilebilir. Örneğin "the dog sleeps" de türetilebilir.

Kaç cümle vardır? Üç değişkenin (article, noun, verb) her biri iki farklı uç değer alabilir. Bu yüzden $2 \cdot 2 \cdot 2 = 8$ farklı dizgi türetilir.

2.3 Gramerin Dili

Bir gramerin dili (language of the grammar), o gramerle türetilebilen bütün dizgilerin kümesidir. Bu örnekte dilin sekiz elemanı vardır: "a cat runs", "a cat sleeps", "the cat runs" ve benzerleri.

2.4 Kurallar, Değişkenler ve Uç Simgeler

Yukarıdaki satırlara bazen kural (rule), bazen üretim (production) denir.

`<verb>`, `<noun>`, `<article>` ve `<sentence>` birer değişkendir (variable). Bu örnekte değişkenler özel bir simgeyle belirtildi. Sonraki örneklerde genelde büyük harf kullanacağız: Harf büyükse değişkendir. Yine de bazen bu tür gösterimler de kullanılabilir.

"sleeps", "runs", "dog", "cat" ise değişken değildir. "sleeps"ten başka bir şey üretemezsiniz. Bir değişken "sleeps" olunca hep "sleeps" kalır. Bu yüzden bunlara uç simge (terminal) denir.

Bir değişken üç tür şey üretebilir: yalnız uç simge, bir değişken dizisi ya da uç simgelerle değişkenlerin karışık bir dizisi. Yani kuralın sağ tarafı yalnız değişkenlerden oluşmak zorunda değildir.

3. İlk Örnek: $S \to aSb \mid \varepsilon$

Şu grameri ele alalım:

$$S \to aSb$$
$$S \to \varepsilon$$

Burada tek değişken S'dir. Bu gösterimde büyük harfler değişkendir. Uç simgeler a ve b'dir. Sağ tarafta boş dizgi (null, ε) de olabilir.

> **Ders sonrası not:** Hoca boş dizgiden "başka bir uç simge" gibi söz etti. Doğrusu, ε bir uç simge değildir, uzunluğu sıfır olan dizgidir. Hocanın biraz sonraki biçimsel tanımda uç simge kümesini yalnız $\{a, b\}$ olarak yazması da bunu gösterir.

3.1 ab Dizgisini Türetmek

Başlangıç değişkeni genellikle en üstte ve en solda yazılır. Bu örnekte ilk satırdaki S'dir. Her zaman ondan başlarız. Elimizde iki kural var. ab istediğimiz için ε kuralıyla başlarsak ab'yi üretemeyiz. Bu yüzden önce $S \to aSb$ kullanılır:

$$S \Rightarrow aSb \Rightarrow ab$$

aSb, istediğimize çok yakındır. Başta a, sonda b var, yalnız fazladan bir büyük S kaldı. Ondan kurtulmak için ikinci kuralı kullanırız. Birinci adımda $S \to aSb$, ikinci adımda $S \to \varepsilon$ kuralı uygulanmıştır.

3.2 Kaç Dizgi Üretilebilir?

Sonsuz sayıda dizgi üretilebilir, çünkü kuralda bir döngü vardır: S'nin sağ tarafında yine S geçer. İstediğiniz kadar S açıp en sonda S'yi boş dizgiye çevirebilirsiniz.

Örneğin $aabb$:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$$

İlk iki adımda $S \to aSb$ kullanılır. Her uygulamada yeni a ve b, öncekilerin içine girer. $aaSbb$, istediğimiz dizgiye çok yakındır. $S \to \varepsilon$ ile S silinince $aabb$ elde edilir.

3.3 Gramerin Dilini Yazmak

Bu grameri G diye adlandıralım. Dilinde ab, $aabb$, $aaabbb$ gibi dizgiler vardır. Desen açıktır: n tane a ve tam aynı sayıda b. Peki ε dilde midir? Evet, çünkü en başta doğrudan $S \to \varepsilon$ kuralını kullanabiliriz. Bu yüzden n sıfır da olabilir:

$$L(G) = \{a^n b^n \mid n \geq 0\}$$

$n = 0$ için ε, $n = 1$ için ab, $n = 2$ için $aabb$, $n = 3$ için $aaabbb$ elde edilir.

4. İki Kısa Gösterim

4.1 Yıldızlı Türetme: $\Rightarrow^*$

$S \Rightarrow^* aaabbb$ yazabiliriz. Buradaki yıldız, bazı adımları yazmadığımız anlamına gelir: Bu değişkenden başlayıp bazı adımlarla sağ taraftaki dizgiye ulaşabilirsiniz. Tam yazılışı şudur:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaabbb$$

Hoca önce bunun "bir ya da daha fazla" adım demek olduğunu söyledi, sonra düzeltti: Doğrusu **sıfır ya da daha fazla** türetme adımıdır. Sıfır adım şöyle olur: Bir w dizgisinden kendisini sıfır adımda elde edersiniz, yani $w \Rightarrow^* w$. Aynı şey bir değişken için de geçerlidir.

4.2 Dikey Çizgi: $|$

Kuralları alt alta yazmak yerine tek satırda yazabiliriz:

$$S \to aSb \mid \varepsilon$$

Dikey çizgi "ya da" gibidir: S bunu ya da şunu üretebilir.

5. Bağlamdan Bağımsız Gramerin Biçimsel Tanımı

Bir bağlamdan bağımsız gramerin içinde dört nesne vardır, yani bir dörtlüdür (4-tuple):

$$G = (V, T, S, P)$$

- V: değişkenler kümesi.
- T: uç simgeler kümesi.
- S: başlangıç değişkeni.
- P: üretimler kümesi.

$S \to aSb \mid \varepsilon$ grameri için bu dört nesne şöyledir:

$$V = \{S\}, \quad T = \{a, b\}, \quad S = S, \quad P = \{S \to aSb,\ S \to \varepsilon\}$$

Sizden bir gramerin biçimsel tanımı istenirse bu dört şeyi böyle, $V = \ldots$, $T = \ldots$ biçiminde vermeniz gerekir. "$S = S$" biraz garip görünebilir. Soldaki S dörtlüdeki başlangıç değişkeni yeridir, sağdaki S ise gramerdeki değişkendir. Bunlar aslında farklı şeylerdir. İsterseniz farklı harf kullanabilirsiniz. Çoğu gramerde S kullanacağız, çünkü İngilizce "start" kelimesinden gelir.

6. Gramerin Dili ve Bağlamdan Bağımsız Dil

Şimdiye kadar gramerin dilinden gayriresmî söz ettik. Biçimsel tanım şudur. Başlangıç değişkeni S olan bir G grameri için:

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$$

Yani dil, S'den başlayıp üretebildiğiniz ve uç simgelerden oluşan w dizgilerinin kümesidir.

Örnek olarak kullandığımız gramerde $L(G) = \{a^n b^n \mid n \geq 0\}$'dır. Çünkü sıfır ya da daha büyük her n için $S \Rightarrow^* a^n b^n$ türetmesi vardır. Hoca bunu önce "n sıfırdan büyük" diye söyledi, hemen ardından "sıfıra eşit de olabilir" diye tamamladı. Başlangıç değişkeninden her zaman belli bir desene ulaştığınızı söyleyebiliyorsanız, dil o desendir.

Bağlamdan bağımsız dil (context-free language) de buna dayanır:

$$L \text{ bağlamdan bağımsızdır} \iff L(G) = L \text{ olan bir bağlamdan bağımsız } G \text{ grameri vardır.}$$

Buradaki iki L farklı şeylerdir. Biri sözünü ettiğimiz dildir. $L(\cdot)$ ise bir fonksiyon gibidir: Ona bir gramer verirsiniz, o da gramerin dilini döndürür. Hangisinin kastedildiği kullanımdan anlaşılır.

Örneğin $\{a^n b^n \mid n \geq 0\}$ bağlamdan bağımsız bir dildir, çünkü onu $S \to aSb \mid \varepsilon$ grameri üretir.

7. Örnek: Çift Uzunluklu Palindromlar

Gramer şu olsun:

$$S \to aSa \mid bSb \mid \varepsilon$$

Dili nedir? Önce birkaç örnek üretip deseni görmeye çalışalım:

$$S \Rightarrow aSa \Rightarrow abSba \Rightarrow abbSbba \Rightarrow abbbba$$

Her adımda solda ne varsa onun tersi sağda oluşuyor. $abbSbba$'da solda abb, sağda onun tersi bba vardır. En sonda S boş dizgi kuralıyla silinir. Buna göre:

$$L(G) = \{ww^R \mid w \in \{a, b\}^*\}$$

Yani dil, a ve b'den oluşan bir w dizgisi ile tersinin birleşimidir. Bunlara palindrom (palindrome) denir. Uzunlukları çifttir, çünkü solda ve sağda aynı sayıda simge vardır.

8. Örnek: Eşleşen Parantezler

Biraz daha karışık bir örnek:

$$S \to aSb \mid SS \mid \varepsilon$$

Yine birkaç adım deneyelim:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaSSbb \Rightarrow aaaSbSbb \Rightarrow aaaSbaSbbb \Rightarrow aaabaSbbb \Rightarrow aaababbb$$

Son iki adım $S \to \varepsilon$ uygulamasıdır (ara adım eklendi).

> **Hocanın vurgusu:** Kurallarda tek ok ($\to$) kullanılır. Türetmeleri ve türetme adımlarını gösterirken kalın ok ($\Rightarrow$) kullanılır.

$aaababbb$ dizgisine bakınca desen ilk anda görünmüyor. Ama a yerine "(" ve b yerine ")" koyunca dizgi `(((()()))` olur. Her açılan parantezin bir kapanan karşılığı vardır. Bu gramer eşleşen parantezleri (matched parentheses) üretir.

8.1 Dili Matematiksel Olarak Yazmak

$n_a(w)$, w dizgisindeki a sayısı olsun. Eşleşme için önce açılan parantez sayısı kapanan parantez sayısına eşit olmalıdır:

$$n_a(w) = n_b(w)$$

Bizim örneğimizde dört a ve dört b vardır. Ama bu koşul yetmez, çünkü sayıları eşit olup da eşleşmeyen dizgiler de vardır. İkinci koşul öneklerle (prefix) ilgilidir:

$$L(G) = \{w \mid n_a(w) = n_b(w) \text{ ve } w\text{'nin her } v \text{ öneki için } n_a(v) \geq n_b(v)\}$$

> **Ek bilgi (derste anlatılmadı):** Derste örneği verilmeyen böyle bir dizgi ba'dır. İçinde birer a ve b vardır, ama ilk öneki olan b'de $n_a < n_b$ olur. Parantezle `)(` biçimindedir ve grameri bu dizgiyi üretmez.

Önek, dizginin başından başlayan bir alt dizgidir. $aaababbb$ için a, aa, aaa, $aaab$, $aaaba$ birer önektir. Hoca $aaabab$ önekini seçti. Burada dört a ve iki b vardır, yani koşul sağlanır. En uzun önek dizginin kendisidir, orada iki sayı eşittir.

Ders sonrası not: Hoca önek koşulunu önce "büyük" diye söyledi, sonra "büyük ya da eşit" dedi. Doğrusu $n_a(v) \geq n_b(v)$'dir. Yalnız "büyük" olsaydı, dizginin kendisi de bir önek olduğu için ilk koşulla çelişirdi. Gramerin dili ile bu küme, uzunluğu 14'e kadar olan bütün dizgilerde tam taramayla karşılaştırıldı ve aynı çıktı.

Bu dili yazmak biraz daha fazla deneyim ve düşünme ister. Ama sonuçta verilen bir gramerin dilini rahatça yazabilmeniz gerekir.

9. Türetme Sırası: En Soldan ve En Sağdan Türetme

Şimdi türetme sırasına (derivation order) bakalım. Gramer şu olsun:

$$S \to AB$$
$$A \to aaA \mid \varepsilon$$
$$B \to Bb \mid \varepsilon$$

Burada beş üretim vardır. Bir dizgide birden fazla değişken varsa önce hangisini değiştireceğimizi seçmemiz gerekir. İki belirli sıra tanımlanır:

- **En soldan türetme (leftmost derivation):** Her adımda en soldaki değişken değiştirilir.
- **En sağdan türetme (rightmost derivation):** Her adımda en sağdaki değişken değiştirilir. Üç ya da dört değişken olsa bile, son değişken seçilir.

Böylece türetme sırası deterministik olur. "En sağdan" deyince, birden çok seçenek olan yerde herkes aynı sırayı kullanır.

9.1 aab İçin En Soldan Türetme

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aaBb \Rightarrow aab$$

Adım adım:

1. Başlangıç değişkeniyle başlarız, başka seçenek yoktur: $S \Rightarrow AB$.
2. İki değişken var, A ve B. En solda A olduğu için $A \to aaA$ uygulanır, B olduğu yerde kalır: $aaAB$.
3. Yine en soldaki A değiştirilir. Bir kez daha aaA yazarsak aab'yi elde edemeyiz. Bu yüzden $A \to \varepsilon$ kullanılır: aaB.
4. Tek değişken kaldı. Dizgide bir b istediğimiz için $B \to Bb$ uygulanır: $aaBb$.
5. Dizgiye neredeyse ulaştık. B'den kurtulmak için $B \to \varepsilon$ kullanılır: aab.

9.2 aab İçin En Sağdan Türetme

$$S \Rightarrow AB \Rightarrow ABb \Rightarrow Ab \Rightarrow aaAb \Rightarrow aab$$

Adım adım:

1. Yine $S \Rightarrow AB$ ile başlarız.
2. Bu kez A yerine en sağdaki B değiştirilir. b elde etmek için $B \to Bb$ seçilir: ABb.
3. Yine iki değişken var, en sağdaki B'dir. Bu kuralı bir daha kullanırsak fazladan b eklenir. Bu yüzden $B \to \varepsilon$ kullanılır: Ab.
4. Tek değişken kaldı: $A \to aaA$ ile $aaAb$.
5. $A \to \varepsilon$ ile aab.

İki türetme aynı mıdır? Hayır. İlk adım aynıdır, ama ikinci adımdan başlayarak farklı dizilimler elde edilir.

10. Türetme Ağaçları

Bu dizilimlere karşılık gelen ağaçlar da vardır: türetme ağacı (derivation tree) ya da ayrıştırma ağacı (parse tree). Hocanın dediği gibi, bunu muhtemelen derleyici konularından biliyorsunuz. Bir derleyici program kodunu derlemeye çalışırken onu ayrıştırır (parse), yani bu ağaçları oluşturur.

Ağaç, türetmeye çok benzer. Tek fark, her adımda dal kullanmamızdır. $S \Rightarrow AB$ yazmak yerine S düğümü (node) çizilir ve A ile B için birer dal açılır.

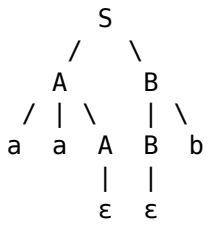
Ağacın yaprakları (leaves) soldan sağa okunursa türetmenin o adımındaki dizi elde edilir. Bu okumaya ürün (yield) da denir. İsterseniz "S, AB'yi üretir" yerine "S yields AB" de diyebilirsiniz.

10.1 aab İçin Ağacı Kurmak

En soldan türetmeyi izleyelim:

1. S'den iki dal: A ve B. Yapraklar: A, B.
2. En soldaki A açılır, B'ye dokunulmaz. A'dan üç dal: a, a, A. Yapraklar: a, a, A, B. Henüz açılmamış dalları da okumaya katarız.
3. Yeni A, ε olur. Yapraklar: a, a, ε, B.
4. Soldan sağa tararız, ilk değişken yaprak B'dir. B'den iki dal: B ve b. Yapraklar: a, a, ε, B, b.
5. Son değişken ε olur. Yapraklar: $a, a, \varepsilon, \varepsilon, b$.

Son ağaç şudur:



Yapraklar birleştirilince $a\,a\,\varepsilon\,\varepsilon\,b$, yani aab çıkar. ε dizgi olmaması demektir, okumaya bir şey katmaz.

Bu örnekte en sağdan türetmenin ağacı da aynıdır. Hoca bunu sizin denemenizi söyledi.

11. Belirsizlik

Bir bağlamdan bağımsız gramer G şu durumda belirsizdir (ambiguous): $L(G)$ içinde, iki farklı türetme ağacı ya da iki farklı en soldan türetmesi olan bir w dizgisi vardır.

"Ambiguous" Türkçede belirsiz, müphem demektir. Ne dediği tam belli değildir, kesin değildir. Gramerin ayrıştırma ağaçları muğlaktır: o da olabilir, bu da olabilir.

Bu neden sorundur? Derleyici yazmak istiyorsanız sorun çıkarır. Çünkü bir kodun tek bir anlamı olsun isteriz. Yazdığınız kodun ne anlama geldiği belirsizse orada bir sorun vardır. Ayrıştırma ağaçları, kodun anlamıdır.

12. Belirsizlik Örneği: Aritmetik İfadeler

Şu gramer matematiksel ifadeler üretir:

$$E \to E + E \mid E \times E \mid (E) \mid a$$

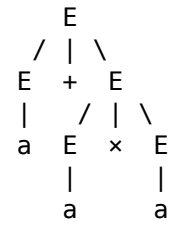
Uç simge a herhangi bir sayıdır. Üretilen dizgiler $a + a \times a + a \times a + a$ gibidir.

12.1 Birinci Ağaç

$a+a\times a$ için en soldan türetme ağacı kuralım. Başlangıç değişkeni E'dir. Burada bir seçimimiz var: Önce $+$ için kuralı ya da önce $\times$ için kuralı kullanabiliriz. Tek değişken olduğu için ikisi de en soldan türetmedir. Bu yüzden "en soldan türetme ağacı" yerine "bir en soldan türetme ağacı" demek gerekir, çünkü birden fazla olacak.

Önce $E \to E + E$ kuralını kullanalım. En soldaki E'nin önünde seçim yoktur, $+$'tan önce tek simge var, o da a. Sonra sıra sağdaki E'ye gelir. $a \times a$ parçasını elde etmek için $E \to E \times E$ uygulanır. Kalan E'ler a olur:

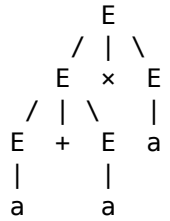
$$E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a$$



12.2 İkinci Ağaç

Yine E ile başlayalım, ama $+$ yerine önce $\times$ kuralını kullanalım. Şimdi en soldaki E'den $+$ üretilmelidir. Sonra soldan sağa bütün E'ler a olur:

$$E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a$$



İki farklı en soldan türetme ağacı var. Belirsizlik budur. Bu G grameri belirsizdir.

12.3 Hocanın Düzelttiği Yanlış

Hoca, Bölüm 10'daki aab örneğine dönüp orada tahtaya yazdığı bir cümleyi sildi. Önce "en sağdan türetme ağacı en soldan olanla aynı, o hâlde bu gramerde belirsizlik yok" demişti.

> **Hocanın vurgusu:** O gramerde belirsizlik yoktur. Ama nedeni en soldan ve en sağdan türetmelerin aynı ağacı vermesi değildir. Nedeni, birden fazla en soldan türetme olmamasıdır. Not aldıysanız o cümleyi düzeltin.
>
> **Ders sonrası not:** Her türetme ağacının bir en soldan ve bir en sağdan türetmesi vardır. Bu yüzden ikisinin aynı ağaca varması belirsizlik hakkında bir şey söylemez. Bölüm 9'daki gramerde uzunluğu 12'ye kadar bütün dizgiler tam taramayla denendi. Her dizginin tek ağacı olduğu görüldü.
>
> **Ek bilgi (derste anlatılmadı):** Tanım, Bölüm 8'deki gramere de uygulanabilir. $S \to aSb \mid SS \mid \varepsilon$ belirsizdir. Çünkü ε dizgisinin hem $S \Rightarrow \varepsilon$ hem de $S \Rightarrow SS \Rightarrow S \Rightarrow \varepsilon$ türetmesi vardır ve bunların ağaçları farklıdır.

12.4 Bu Neden Sorun?

$a = 2$ alalım. Bu dili derleyen bir hesap makinesi yazdığınızı ve sonucu döndürdüğünü düşünün.

- **Birinci ağaç:** $2 + (2 \times 2) = 2 + 4 = 6$.
- **İkinci ağaç:** $(2 + 2) \times 2 = 4 \times 2 = 8$.

İki sonuç eşit değildir. Belirsizlik varsa dizgilerin anlamı deterministik değildir, derleyicinin kullandığı sıraya göre değişir. Bilgisayar dillerinde normalde işlem önceliği (precedence) vardır: Çarpma toplamadan önce yapılır. Bu bir şekilde gramere de konur. Oysa bu gramerde öncelik yoktur. Kurallar arasında bir sıra yoktur, derleyici başta $+$ ya da $\times$ kuralını seçebilir. Ayrıştırma ağaçlarını kullanan derleyici gibi uygulamalarda iki farklı türetme ağacı bu yüzden sorun çıkarır.

13. Tanıma Dönüş: Ağaç mı, Türetme mi?

Aradan sonra hoca belirsizlik tanımını bir daha açtı. Tanımda "iki farklı türetme ağacı ya da iki farklı en soldan türetme" deniyor. Bu ayrım önemlidir.

> **Hocanın vurgusu:** Ağaçlarla bakıyorsanız ağaçların en soldan olması gerekmez. İki farklı türetme ağacı varsa gramer belirsizdir. Türetmelerle karar vermeye çalışıyorsanız türetmeler en soldan olmalıdır. Buna dikkat edin.

Türetme ağacı, çizdiğimiz ağaçtır. Türetme ise S'den başlayıp $S \Rightarrow \ldots$ biçiminde yazılan dizidir.

14. Belirsizlik Örneği: Sarkan Else

İkinci gerçek örnek programlamadaki if-else ifadesidir. Gramer şöyledir:

- `<if_stmt>` $\rightarrow$ **if** `<expr>` **then** `<stmt>`
- `<if_stmt>` $\rightarrow$ **if** `<expr>` **then** `<stmt>` **else** `<stmt>`
- `<stmt>` $\rightarrow$ `<if_stmt>`

Köşeli parantezliler değişkendir. Uç simgeler **if**, **then** ve **else**'tir. Üçüncü kuralı hoca ağacı çizerken ekledi: Bir if ifadesi de bir statement olabilmelidir. Hocaya göre kullandığı kaynakta bu kural eksikti. Bu kural olmadan iç içe if türetilemez.

Dizgi şu olsun:

```
if E1 then if E2 then S1 else S2
```

Bu dizgi için iki farklı ağaç kurulabilir.

Birinci ağaç. Önce kısa kuralı kullanırız: **if** E **then** S. Sonra S'yi üçüncü kuralla if ifadesine çevirir, oradan uzun kuralı uygularız. E'ler E_1 ve E_2, S'ler S_1 ve S_2 olur. Else içteki if'e bağlanır:

```
if E1 then ( if E2 then S1 else S2 )
```

İkinci ağaç. Önce uzun kuralı kullanırız: **if** E **then** S **else** S. İlk S'yi üçüncü kuralla if ifadesine çevirir, oradan kısa kuralı uygularız. Else dıştaki if'e bağlanır:

```
if E1 then ( if E2 then S1 ) else S2
```

Bu yüzden bu gramer belirsizdir. Bu dizginin ağaçları sayılınca da tam iki ağaç çıktı.

14.1 Diller Ne Yapıyor?

Bazı programlama dilleri, örneğin Perl, bu yüzden böyle bir if yazımına izin vermez ve süslü parantez (curly braces) zorunlu tutar. Süslü parantez olunca belirsizlik çözülür: Else'in hangi if'e ait olduğu sorunu kalkar.

C gibi bazı diller ise süslü parantezsiz if-else'e izin verir. O zaman ne yazdığınıza dikkat etmeniz gerekir. Bu, belirsizlikle ilgili gerçek ve sorunlu bir durumdur.

> **Ek bilgi (derste anlatılmadı):** C'de bu belirsizlik, else'in en yakın eşleşmemiş if'e bağlanmasıyla çözülür. Bu not için gcc ile küçük bir programda denendi: `if(0) if(1) x=1; else x=2;` satırı x'i değiştirmedi, yani else içteki if'e bağlandı.

15. Belirsizliği Gidermek

Belirsizlikten nasıl kurtuluruz? Bu probleme bağlıdır, üzerinde çalışmanız gerekir. Perl'de süslü parantezi zorunlu yapmak bir yoldur. Önceki aritmetik örneğinde ise başka bir yol vardır.

Belirsiz gramerin yerine, aynı dili üreten eşdeğer ve belirsiz olmayan (unambiguous) bir gramer kullanılır:

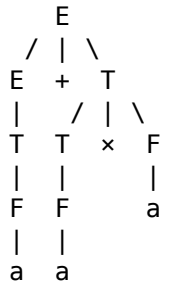
$$E \to E + T \mid T$$
$$T \to T \times F \mid F$$
$$F \to (E) \mid a$$

Bu gramer birinci ve ikinci terimi birbirinden ayırır. Bunu nasıl bulduklarını düşünmek biraz çalışma ister. Nasıl işlediğine $a + a \times a$ üzerinde bakalım.

15.1 $E \to E + T$ ile Başlamak

İki seçeneğimiz var: $E + T$ ya da T. Önce $E + T$'yi deneyelim. Soldaki E, a olur. Hoca burada ara adımları geçti, zincir şudur: $E \Rightarrow T \Rightarrow F \Rightarrow a$ (ara adım eklendi). Sağdaki T, $T \times F$ olur. F doğrudan a olabilir. T ise önce F'ye, sonra a'ya gider:

$$E \Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow a + T$$
$$\Rightarrow a + T \times F \Rightarrow a + F \times F \Rightarrow a + a \times F \Rightarrow a + a \times a$$



15.2 $E \to T$ ile Başlamak

Farklı bir türetme var mı diye T ile başlamayı deneyelim. T, $T \times F$ olur, F de a olur. Geriye T'den $a + a$ üretmek kalır. $+$ üretmek için E gerekir. T'den F'ye, F'den de ancak parantez içinden E'ye gidebiliriz. Dizgimizde ise parantez yok. Bu yol tıkanır.

Yani bu dizgi için tek bir yol vardır, ayrıştırma ağacı tektir. Çarpma ağaçta toplamadan daha derinde üretildiği için önce yapılır.

> **Ders sonrası not:** Hoca bu kısımda dizgiyi bir yerde "a plus a times b" diye söyledi, doğrusu $a + a \times a$'dır. Bu gramerde b uç simgesi yoktur. Ayrıca uzunluğu 11'e kadar bütün dizgiler tam taramayla denendi: Bu gramer ile Bölüm 12'deki gramer aynı dizgileri üretiyor ve yeni gramerde her dizginin tek ağacı var. Bu sınırlı bir doğrulamadır, genel ispat değildir.

16. Giderilemeyen Belirsizlik: Doğası Gereği Belirsiz Diller

Bu iyi durumlarda eşdeğer belirsiz olmayan bir gramer bulabildik. Ama bu her zaman mümkün değildir. Bazen belirsizliği çözemezsiniz. Bu dillere doğası gereği belirsiz diller (inherently ambiguous languages) denir: Bu dili üreten **her** gramer belirsizdir. Belirsiz olmayan bir gramer bulamazsınız.

16.1 L_1 Dili

$$L_1 = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}, \quad n, m \geq 0$$

Bu dil ne diyor? Birkaç duruma bakalım:

- Soldaki kümede $n = 0$, $m \neq 0$ olursa c gibi dizgiler çıkar.
- Soldaki kümede $m = 0$, $n \neq 0$ olursa ab gibi dizgiler çıkar.
- Sağdaki kümede $m = 0$, $n \neq 0$ olursa a gibi dizgiler çıkar.
- Sağdaki kümede $n = 0$ iken bc gibi dizgiler çıkar.

Koşul şudur: b'lerin sayısı ya a'ların sayısına ya da c'lerin sayısına eşit olmalıdır. Örneğin $aabbc$'de a ve b sayıları eşittir. $abbcc$'de b ve c sayıları eşittir.

16.2 L_1 İçin Bir Gramer

$$S \to S_1 \mid S_2$$
$$S_1 \to S_1 c \mid A$$
$$A \to aAb \mid \varepsilon$$
$$S_2 \to aS_2 \mid B$$
$$B \to bBc \mid \varepsilon$$

S_1 kolu soldaki kümeyi üretir: c'leri yazar, sonra A ile eşit sayıda a ve b üretir. S_2 kolu sağdaki kümeyi üretir: a'ları yazar, sonra B ile eşit sayıda b ve c üretir.

Hoca bir dizgi türetmeye başladı: $S \Rightarrow S_1 \Rightarrow S_1 c \Rightarrow S_1 cc \Rightarrow Acc \Rightarrow aAbcc$. Burada zamanın yetmediğini söyleyerek durdu ve L_1 ile L_2'yi evde çalışmanızı istedi.

> **Ders sonrası not:** Türetme yarım kaldı. Örneğin $aabbcc$ iki koldan da türetilir: $S \Rightarrow S_1 \Rightarrow S_1 c \Rightarrow S_1 cc \Rightarrow Acc \Rightarrow aAbcc \Rightarrow aaAbbcc \Rightarrow aabbcc$ ve $S \Rightarrow S_2 \Rightarrow aS_2 \Rightarrow aaS_2 \Rightarrow aaB \Rightarrow aabBc \Rightarrow aabbBcc \Rightarrow aabbcc$. Bu yüzden bu gramer belirsizdir. Uzunluğu 12'ye kadar tam tarama, gramerin dilinin L_1 olduğunu ve iki ağacı olan dizgilerin tam olarak $a^n b^n c^n$ biçimindekiler olduğunu gösterdi. Bu tarama yalnız bu gramer içindir. L_1'in her gramerinin belirsiz olduğu ayrı bir teoremdir.

16.3 L_2 Dili

Hoca ikinci örnek olarak şunu verdi:

$$L_2 = \{a^n b^n c^n \mid n > 0\}$$

ve bunun da belirsiz olduğunu, üreten bir gramer bulunamayacağını söyledi.

> **Ders sonrası not:** Hoca L_2'yi L_1 ile aynı başlık altında "belirsiz" diye verdi. Doğrusu, L_2 bağlamdan bağımsız değildir, yani onu üreten hiçbir bağlamdan bağımsız gramer yoktur. Bu, bağlamdan bağımsız diller için pompalama önsavıyla (pumping lemma) ispatlanır. "Doğası gereği belirsiz" kavramı yalnız bağlamdan bağımsız diller için kullanılır. Bu yüzden L_2 doğası gereği belirsiz dil örneği değildir. Hocanın "gramer bulunamaz" sözü ise doğrudur.

> **Ders sonrası not:** Hoca önce bu dillere "ambiguous languages" dedi, sonra "inherently ambiguous" terimini kullandı. Doğru terim "inherently ambiguous"tır. Gramer belirsiz ya da belirsiz olmayan olabilir. Dil için ise doğası gereği belirsiz denir.

Soru: Tahtada L_1 ve L_2'yi önce belirsiz diye gösterdik, aşağıda ise "see that they are unambiguous" yazıyor. Hangisi doğru? **Cevap:** Bu dillerde ne yaparsanız yapın, belirsiz olmayan bir gramer yazamıyorsunuz. Evde bunu görün demek istedim, derste vaktimiz yetmedi.

17. Bir Dilin Bağlamdan Bağımsız Olduğunu Göstermek

Ders sonunda hoca, önümüzdeki konularla bağlantıyı şöyle kurdu. Bir dilin bağlamdan bağımsız olduğunu göstermek için onun gramerini yazmanız gerekir. Bir dilin bağlamdan bağımsız olmadığını göstermek için ise bağlamdan bağımsız diller için pompalama önsavını bilmeniz gerekir. Bazı soruları çözmek için de aşağı itmeli otomat bilmek gerekir. Bu iki konuyu henüz görmedik.

Dersin Toparlaması

Bu derste düzenli olmayan $a^n b^n$ ve ww^R gibi dillerin, bağlamdan bağımsız gramerlerle tanımlanabildiğini gördük. Bir gramer değişkenlerden, uç simgelerden, başlangıç değişkeninden ve üretimlerden oluşur. Dili, başlangıç değişkeninden türetilebilen bütün uç simge dizgileridir. Verilen bir gramerin dilini bulmak için birkaç dizgi türetip deseni küme gösterimiyle yazdık.

Sonra türetmelerin sırasını en soldan ve en sağdan diye belirledik ve türetmeleri ağaçlarla gösterdik. Bir dizginin iki farklı ağacı ya da iki farklı en soldan türetmesi varsa gramer belirsizdir. Aritmetik ifadeler ve sarkan else örnekleri bunun derleyicide neden sorun olduğunu gösterdi. Belirsizlik bazen gramer değiştirerek ya da dil kuralıyla giderilir, bazen de hiç giderilemez. Sıradaki konular aşağı itmeli otomatlar ve bağlamdan bağımsız diller için pompalama önsavıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.